# CS 300 – Chapter 7 Notes

## Heaps and Priority Queues

Jose de Lima

April 21, 2026

## 1 7.1 Heaps: The Big Idea

A **heap** is a binary tree specialized for one job: hand me the max (or min) element, fast, and let me insert or remove in logarithmic time. It is not a BST – it deliberately gives up ordered iteration to gain simplicity and speed on that narrow interface. Where a BST balances every subtree to permit arbitrary search, a heap only cares that *the root is the extremum.*

**Definition — Max-heap property**

A binary tree satisfies the *max-heap property* if, for every node $n$:

$$n.\text{key} \;\geq\; n.\text{left.key} \quad \text{and} \quad n.\text{key} \;\geq\; n.\text{right.key}$$

(where absent children are ignored). By transitivity, the root's key is $\geq$ every descendant – *the root holds the maximum.*

The **min-heap** is the mirror image: swap $\geq$ for $\leq$. The root holds the minimum. Everything we say about max-heaps inverts cleanly.

### What a heap is *not*

**Gotcha**

A heap is **not** a BST. The left and right subtrees are not sorted relative to each other – only relative to their parent. You cannot in-order traverse a heap and get sorted output. You cannot binary-search a heap for an arbitrary key. A heap is optimized for *extremum access*, nothing else.

A quick check: in the BST from chapter 6, `node->left < node < node->right`. In a max-heap, `node->left` $\leq$ `node` and `node->right` $\leq$ `node`, but `node->left` and `node->right` are unordered with respect to each other. Two completely different invariants.

### Why not just use a sorted list?

We already have structures that hand over the max in $O(1)$:

- A sorted array: max is at the last index. Remove: $O(1)$. Insert: $O(n)$ to find the spot and shift.
- A BST: max is at the rightmost node. Insert/remove: $O(h) = O(\log n)$ balanced, $O(n)$ skewed.
- A sorted linked list: max at the tail. Insert: $O(n)$.

A heap beats all of these for the "give me the max, keep handing me max, occasionally insert new stuff" workload:

| Structure | Peek max | Remove max | Insert | Notes |
|---|---|---|---|---|
| Sorted array | $O(1)$ | $O(1)$ | $O(n)$ | shifts on insert |
| Sorted linked list | $O(1)$ | $O(1)$ | $O(n)$ | scan on insert |
| Unsorted array | $O(n)$ | $O(n)$ | $O(1)$ | rescan per op |
| Balanced BST | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ | overkill |
| **Max-heap** | $\mathbf{O(1)}$ | $\mathbf{O(\log n)}$ | $\mathbf{O(\log n)}$ | perfect fit |

The heap trades away ordered iteration and arbitrary search (which we don't need for this workload) to get $O(\log n)$ insert with $O(1)$ peek.

## The canonical use case: priority queues

A **priority queue** is an abstract data type that supports `insert(item, priority)`, `extractMax()` (or `extractMin()`), and `peek()`. Heaps are the standard implementation. Real-world examples:

- **OS process schedulers:** ready processes live in a max-heap keyed by priority; the scheduler extracts the highest-priority ready process on each quantum.
- **Online support queues:** customers live in a min-heap keyed by "order of arrival + urgency bonus" – whoever has the lowest number (highest effective priority) gets the next agent. This is a common running example in textbooks.
- **Dijkstra / A* / Prim:** graph algorithms repeatedly extract the next "cheapest" frontier node. A min-heap makes those $O((V + E) \log V)$.
- **Event simulations:** an event-loop heap keyed by simulated timestamp lets you pop "next event" efficiently.
- **Top-$k$ queries:** maintain a min-heap of size $k$; for each incoming item, if it's bigger than the heap's min, pop and push. Net $O(n \log k)$ instead of $O(n \log n)$ for "top 10 from a stream of a million."

## The two core operations

Every heap algorithm reduces to two primitives. In a max-heap:

**Definition — Percolate up / Percolate down**

**Percolate up** (also "bubble up" or "sift up"): take a node whose key might be too large for its parent. While the node's key $>$ its parent's key, swap with parent. Used after insertion.
**Percolate down** ("bubble down" / "sift down"): take a node whose key might be too small for its children. While the node has a child with a larger key, swap with the larger child. Used after the root is removed.

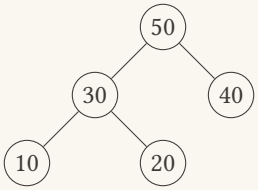
Both run in $O(\log n)$ because the tree is kept **complete** – every level is full except possibly the last, which fills left-to-right. Completeness forces height $= \lfloor \log_2 n \rfloor$, so a single root-to-leaf path is logarithmic.

## Insert: percolate up

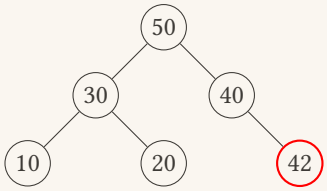
1. Place the new node in the next available slot (leftmost position on the deepest level, extending to a new level if the current one is full). This preserves completeness.

2. Compare the new node to its parent. If it's larger, swap. Repeat until the parent is larger or we reach the root.

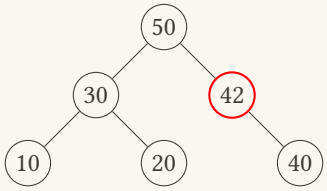
Walk-through: insert 42 into




Step 1: attach 42 as right child of 40 (first empty slot):




Step 2: $42 > 40$? Yes. Swap.




Step 3: $42 > 50$? No. Stop. Heap property restored.

### Remove (max): replace-and-percolate-down

1. Save the root's value (that's the answer we return).
2. Take the *last* node (rightmost leaf of the deepest level) and put it at the root. Delete the now-duplicate last slot. This preserves completeness but usually breaks the heap property at the root.
3. Percolate down: compare the new root against its two children. If the larger child is greater than the node, swap with that child. Repeat until no larger child exists or the node reaches a leaf.

The "replace with last then percolate down" dance is the whole algorithm. You're not finding a replacement by careful search – you grab the structural last node to keep the tree complete, and then let percolate-down sort out the resulting local violation.

**Key idea**

Heaps are the cleanest example in this course of *trading generality for speed*. They give up ordered search, in-order traversal, and range queries – capabilities a BST provides – to get a guaranteed $O(1)$ peek and $O(\log n)$ insert/extract with a tiny implementation. Whenever you only need the extremum (and not ordered iteration), reach for a heap first.

### In the C++ STL

`std::priority_queue` is a max-heap by default, built on top of `std::vector`:

```
#include <queue>
std::priority_queue<int> pq;        // max-heap of ints
pq.push(30); pq.push(50); pq.push(40);
std::cout << pq.top();              // 50
pq.pop();
std::cout << pq.top();              // 40
```

For a min-heap, pass `std::greater`:

```
std::priority_queue<int, std::vector<int>, std::greater<int>> minpq;
```

Section 7.2 will show how the tree is actually stored – surprise, not with pointers. Sections 7.3 and 7.4 formalize `percolateUp` and `percolateDown`. Section 7.5 builds the priority-queue ADT on top.

## 2 7.2 Heaps Stored in Arrays

Here's the twist: the binary tree in chapter 6 uses a `TreeNode` struct with `left`, `right`, (`parent`) pointers per node. A heap uses **nothing of the sort.** A heap is almost always stored as a plain array, with all parent/child relationships computed by *index arithmetic*. No pointers, no allocations per node, no cache misses on node traversal. This is the single biggest reason the heap is the go-to priority queue in practice.

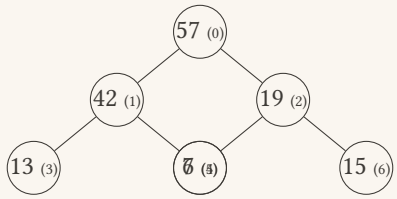
**Definition — Array representation of a complete binary tree**

Number the nodes level by level, left to right, starting from the root at index 0. For a node at index $i$:

$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor \qquad \text{leftChild}(i) = 2i+1 \qquad \text{rightChild}(i) = 2i+2$$

These formulas require the tree to be *complete* – every level full except possibly the last, which fills left-to-right. That's exactly the property heap operations preserve.

### Example




stored as:

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| Value | 57 | 42 | 19 | 13 | 6 | 7 | 15 |

Check: `parent(4) = (4-1)/2 = 1`, so parent of 6 is 42 – correct. `leftChild(2) = 5`, so left child of 19 is 7 – correct.

**Key idea**

A *complete* binary tree has no gaps, so every array slot corresponds to an actual node. That's why the index formulas work – if the tree had missing internal nodes, you'd need sentinels or a different encoding. This is the crucial reason heap insert and remove are written to *preserve completeness structurally first* and fix the heap-property violation second.

## Why arrays beat pointers here

| Pointer-based tree | Array-based heap |
|---|---|
| 3 pointers per node (24 B on 64-bit) | No per-element overhead |
| Cache miss per child jump | Cache-friendly (children are 1–2 cells ahead) |
| `new`/`delete` per insert/remove | `vector` grows amortized $O(1)$ |
| Works for arbitrary tree shapes | Requires completeness |

For a heap of a million ints: pointer representation is $\approx 28$ MB of overhead beyond the values themselves; array representation is $\approx 0$ overhead. The array form is also a contiguous memory block, so sequential scans (like heapsort) are prefetch-friendly.

## Percolate-up, array version

Starting at some index `i` where the heap property might be broken upward, walk toward the root swapping with parent while the child is larger:

```
void maxHeapPercolateUp(int i, std::vector<int>& h) {
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (h[i] <= h[parent]) return;          // heap property satisfied
        std::swap(h[i], h[parent]);
        i = parent;                              // continue from new position
    }
}
```

The loop runs at most $\lfloor \log_2 n \rfloor$ times because each iteration halves the index. That's the $O(\log n)$ insert bound.

## Percolate-down, array version

Starting at some index `i` (usually 0, after the root was replaced), walk toward the leaves swapping with the larger child while a child is larger than the node:

```
void maxHeapPercolateDown(int i, std::vector<int>& h) {
    int n = static_cast<int>(h.size());
    while (2 * i + 1 < n) {                      // at least one child exists
        int left  = 2 * i + 1;
        int right = 2 * i + 2;
        int biggest = left;
        if (right < n && h[right] > h[left]) biggest = right;
        if (h[i] >= h[biggest]) return;          // heap property satisfied
        std::swap(h[i], h[biggest]);
        i = biggest;
    }
```

```
}
```

Watch the three subtleties:

- **Check left first.** If `2*i+1 >= n`, `i` is a leaf and we're done.
- **Right might not exist.** That's why the right comparison is guarded by `right < n`.
- **Swap with the *larger* child, not an arbitrary child.** If you swap with the smaller, the larger child becomes the new root and you've created a new violation below.

The loop runs at most $\lfloor \log_2 n \rfloor$ times (each iteration doubles the index). That's the $O(\log n)$ extract-max bound.

**Gotcha**

Off-by-one bugs are the dominant failure mode here. Triple-check your index formulas – $(i-1)/2$ for parent uses integer division, which works because the formula is exact for both left ($i = 2p+1$) and right ($i = 2p+2$) children. Writing $i/2 - 1$ or $i/2$ will pass for some inputs and silently corrupt for others.

## Insert and extract, array versions

```
void maxHeapInsert(std::vector<int>& h, int value) {
    h.push_back(value);                          // step 1: preserve
        completeness
    maxHeapPercolateUp(h.size() - 1, h);         // step 2: restore heap
        property
}

int maxHeapExtract(std::vector<int>& h) {
    int top = h[0];
    h[0] = h.back();                             // move last to root
    h.pop_back();                                // shrink
    if (!h.empty()) maxHeapPercolateDown(0, h);
    return top;
}
```

Two functions, each $O(\log n)$ worst case, each three lines of logic. No tree allocations, no node structs, no parent pointers. The index formulas are the whole trick.

**Aside**

**Zero-indexed vs. one-indexed.** Some textbooks and papers use index 1 as the root and leave index 0 unused. Then $\text{parent}(i) = i/2$, $\text{left}(i) = 2i$, $\text{right}(i) = 2i+1$ – slightly cleaner math. C++ code defaults to zero-indexed because `std::vector` is zero-indexed. Don't mix conventions in one codebase; pick one and stick with it. This course uses zero-indexed.

## 3 7.3 Heapsort

Heapsort is the sorting algorithm you get almost for free once you have a heap. The idea: turn an array into a max-heap, then repeatedly extract the max to the end of the array. You end up with an array sorted in ascending order, using *no extra memory beyond the input array itself.*

Two phases:

1. **Heapify** the input array in place – transform it into a valid max-heap.
2. **Sort down**: swap `arr[0]` (the max) with the last unsorted slot, shrink the heap by one, and percolate the new root down. Repeat until the heap is empty.

Both phases use one primitive: `maxHeapPercolateDown` from section 7.2.

### Heapify: bottom-up heap construction

Given an arbitrary array, how do we make it a max-heap? Naive approach: insert all elements one by one, each an $O(\log n)$ operation – total $O(n \log n)$. There is a better way.

**Definition — Heapify**

Call `percolateDown` on every internal node in reverse index order, from the last internal node down to the root. Leaves are trivially heaps (no children to violate anything), and each percolate-down restores the heap property over its subtree assuming the children are already valid heaps.

The last internal node is at index $\lfloor n/2 \rfloor - 1$, because nodes at index $\geq \lfloor n/2 \rfloor$ are leaves (their first child index $2i + 1$ is out of range).

```
void heapify(std::vector<int>& a) {
    for (int i = static_cast<int>(a.size()) / 2 - 1; i >= 0; --i)
        maxHeapPercolateDown(i, a);
}
```

### Why heapify is $O(n)$, not $O(n \log n)$

This is the cleverness of bottom-up construction, and it catches most students off-guard on exams.

Percolate-down at index $i$ does work proportional to the *height of the subtree rooted at* $i$, not the total height of the tree. A leaf has height 0 (does no work). A parent of leaves has height 1 (at most 1 swap). The root has height $\log n$. Summed over all nodes:

$$\sum_{i=0}^{\lfloor \log n \rfloor} (\text{nodes at height } i) \cdot i = \sum_{i=0}^{\lfloor \log n \rfloor} \frac{n}{2^{i+1}} \cdot i = O(n)$$

The geometric series in $i/2^i$ converges to a constant. Half the nodes are leaves (do 0 work), a quarter do at most 1 swap, an eighth do at most 2 swaps, and so on. The totals balance out to $O(n)$.

**Key idea**

This is the right mental image: bottom-up heapify cheats the worst case because *most of the nodes are near the bottom where their percolate-down has nothing to do.* Repeated insertion is $O(n \log n)$ because it forces every element through a full height's worth of potential swaps. Same final data structure, different total cost. Prefer heapify over "insert one by one" whenever you have all the data up front.

## The sort-down phase

After heapify, `a[0]` holds the maximum. The classic "swap with last, shrink heap, percolate down" trick used in `maxHeapExtract` (section 7.2) – but we keep the extracted value in the array instead of returning it:

```
void heapSort(std::vector<int>& a) {
    heapify(a);

    int n = static_cast<int>(a.size());
    for (int end = n - 1; end > 0; --end) {
        std::swap(a[0], a[end]);            // move current max to its final
            spot
        maxHeapPercolateDown(0, a, end);    // heap is now indices [0, end)
    }
}
```

Notice the overloaded `percolateDown` here takes an explicit `size` parameter – after each swap, the last slot is "sorted off" and the heap shrinks. You're not physically removing elements; you're just treating the suffix as already-sorted.

## Walk-through

Sort `[11, 21, 12, 13, 19, 15]`.

Step 1 – heapify, starting at index $\lfloor 6/2 \rfloor - 1 = 2$:

| | | | | | | |
|---|---|---|---|---|---|---|
| Before: | 11 | 21 | 12 | 13 | 19 | 15 |
| percolateDown(2): node 12 vs children (15) – swap 12, 15 | 11 | 21 | **15** | 13 | 19 | **12** |
| percolateDown(1): node 21 vs children (13, 19) – already biggest | 11 | 21 | 15 | 13 | 19 | 12 |
| percolateDown(0): node 11 vs (21, 15) – swap with 21; then 11 vs (13, 19) – swap with 19 | **21** | **19** | 15 | 13 | **11** | 12 |

Step 2 – sort down:

| | | | | |
|---|---|---|---|---|
| | 21 | 19 | 15 | 13 |
| swap a[0]=21, a[5]=12; heap size 5; percolate 12 → children (19, 15) swap 12,19; then (13,11) stay | | | | |
| | 19 | 13 | 15 | 12 |
| swap a[0]=19, a[4]=11; size 4; percolate 11 → (13, 15) swap 11, 15; then (12) stay | | | | |
| | 15 | 13 | 11 | 12 |
| swap a[0]=15, a[3]=12; size 3; percolate 12 → (13, 11) swap 12, 13 | | | | |
| | 13 | 12 | 11 | **15** |
| swap a[0]=13, a[2]=11; size 2; percolate 11 → (12) swap | | | | |
| | 12 | 11 | **13** | 15 |
| swap a[0]=12, a[1]=11; size 1; done | | | | |
| | 11 | **12** | 13 | 15 |

Final: `[11, 12, 13, 15, 19, 21]`. Sorted ascending.

## Cost analysis

| Phase | Per step | Total |
|---|---|---|
| Heapify | up to $O(\log n)$ per node | $O(n)$ total (the geometric sum) |
| Sort-down | $O(\log n)$ per extraction | $O(n \log n)$ |
| **Heapsort** | | $\mathbf{O(n \log n)}$ |

Space: $O(1)$ auxiliary. Heapsort is the canonical *in-place* $O(n \log n)$ sort – it rearranges the input array without needing a second array.

## Heapsort vs. quicksort vs. mergesort

| Algorithm | Worst case | Space | Stable? |
|---|---|---|---|
| Quicksort | $O(n^2)$ | $O(\log n)$ stack | no |
| Mergesort | $O(n \log n)$ | $O(n)$ | yes |
| Heapsort | $O(n \log n)$ | $O(1)$ | no |

Heapsort's killer feature is the combination of $O(n \log n)$ *worst case* and $O(1)$ auxiliary space. Quicksort has a worst case of $O(n^2)$; mergesort needs $O(n)$ extra memory. In practice, quicksort is usually fastest due to cache effects, but heapsort is picked when you need worst-case guarantees (embedded systems, real-time deadlines, adversarial inputs).

**Gotcha**

Heapsort is **not stable**: equal keys may swap order during the percolate steps. If stability matters (sorting records by one field without disturbing existing order on another), use mergesort or a stable quicksort variant, not heapsort.

**Aside**

`std::sort_heap` and `std::make_heap` in `<algorithm>` give you the two phases of heapsort. `std::partial_sort` uses a heap-based algorithm to find and sort the top $k$ in $O(n \log k)$. None of these are the full `std::sort`, which typically uses introsort (quicksort that falls back to heapsort when recursion depth gets bad) – precisely because heapsort's worst-case guarantee is the backup plan when quicksort starts to degrade.

# 4 7.4 The Priority Queue ADT

Section 7.1 motivated heaps with the priority-queue use case. This section formalizes the *abstract data type* (ADT) and shows why heaps are the near-universal implementation. An ADT is a specification of operations and their contracts – it tells you *what* a data structure does, not *how* it's implemented.

**Definition — Priority queue**

A **priority queue** (PQ) is a collection where each element has a priority. The defining operations:

- `push(item)` – insert an element.
- `pop()` – remove and return the highest-priority element.
- `peek()` – return (but don't remove) the highest-priority element.
- `isEmpty()` / `size()` – the obvious.

"Highest priority" can mean max-key or min-key depending on whether you want a max-PQ or min-PQ.

Notice what the ADT *doesn't* promise: arbitrary search, ordered iteration, range queries, positional access. A priority queue is FIFO ordered only among items of equal priority (that's the "queue" part), and even that is often not guaranteed unless the implementation makes a point of preserving it.

## Two conventions for "priority"

A priority queue implementation has to answer: does the caller supply a priority alongside each item, or does the item carry its own priority?

**Separate-priority API:**

```
pq.pushWithPriority(jobPointer, priorityNumber);
auto best = pq.pop();   // returns the stored item (not the priority)
```

Natural when the same object might live in multiple PQs with different priorities (e.g., a process could be in a CPU scheduler keyed by niceness and in an I/O scheduler keyed by deadline).

**Self-priority API:**

```
struct Customer { std::string name; int waitScore; };
// ordering comparator compares on waitScore
pq.push(customer);
auto next = pq.pop();
```

Natural when priority is a property of the item itself. This is what `std::priority_queue` does – it takes a comparator and compares items directly.

Both are just different ways to express the same underlying concept. The self-priority API typically requires a comparator; the separate-priority API stores `(priority, item)` pairs and compares on the first component.

## Contract summary

| Operation | Pre-condition | Post-condition |
|---|---|---|
| `push(x)` | heap has room (or is resizable) | $x$ is in the PQ; size $+1$ |
| `pop()` | PQ is non-empty | returns the max/min; that item is removed; size $-1$ |
| `peek()` | PQ is non-empty | returns the max/min; PQ unchanged |
| `isEmpty()` | none | true iff size $= 0$ |
| `size()` | none | returns non-negative count |

Violating the pre-condition (`pop` on empty) is undefined behavior in C++'s `std::priority_queue`; Java's `PriorityQueue` returns null or throws `NoSuchElementException` depending on which method you call (`poll` vs. `remove`). Always check `isEmpty()` first, or wrap the call in a try/check.

## Why heaps are the default implementation

| Backing structure | push | pop (max) | peek | extra notes |
|---|---|---|---|---|
| Unsorted array/list | $O(1)$ | $O(n)$ | $O(n)$ | peek rescans |
| Sorted array | $O(n)$ | $O(1)$ | $O(1)$ | insert shifts |
| Sorted linked list | $O(n)$ | $O(1)$ | $O(1)$ | insert scans |
| Balanced BST | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ | overkill, higher constants |
| **Binary heap** | $O(\log n)$ | $O(\log n)$ | $O(1)$ | tight constants, cache-friendly |
| Fibonacci heap | $O(1)$ amort. | $O(\log n)$ amort. | $O(1)$ | complex, big constants |

The binary heap is the *simplest* structure that gives log-factor bounds on both push and pop with $O(1)$ peek. It's also the one where the implementation is small, cache-friendly, and easy to reason about. Fibonacci heaps have better asymptotic bounds but their constant factors are so bad they're only a win at sizes real problems don't usually reach.

## Mapping PQ operations to heap operations

| PQ op | Heap op (section 7.2) | Cost |
|---|---|---|
| `push(x)` | `maxHeapInsert`: push to back, percolate up | $O(\log n)$ |
| `pop()` | `maxHeapExtract`: swap root/last, pop, percolate down | $O(\log n)$ |
| `peek()` | return `h[0]` | $O(1)$ |
| `isEmpty()` | return `h.empty()` | $O(1)$ |
| `size()` | return `h.size()` | $O(1)$ |

A priority queue is, in essentially all real codebases, just a thin wrapper around a heap's array.

## Implementation sketch in C++

```
template <typename T, typename Cmp = std::less<T>>
class PriorityQueue {
    std::vector<T> h_;
    Cmp cmp_;   // cmp_(a, b) == true means "a has lower priority than b"
                // For a max-heap with std::less, cmp_(a, b) is a < b,
                // which is what we percolate up on.

    void percolateUp(std::size_t i) {
        while (i > 0) {
            std::size_t p = (i - 1) / 2;
            if (!cmp_(h_[p], h_[i])) return;        // parent is larger
                already
```

```
            std::swap(h_[p], h_[i]);
            i = p;
        }
    }
    void percolateDown(std::size_t i) {
        std::size_t n = h_.size();
        while (2 * i + 1 < n) {
            std::size_t l = 2 * i + 1, r = 2 * i + 2, best = l;
            if (r < n && cmp_(h_[l], h_[r])) best = r;
            if (!cmp_(h_[i], h_[best])) return;
            std::swap(h_[i], h_[best]);
            i = best;
        }
    }
public:
    bool empty() const { return h_.empty(); }
    std::size_t size() const { return h_.size(); }
    const T& peek() const { return h_.front(); }

    void push(T value) {
        h_.push_back(std::move(value));
        percolateUp(h_.size() - 1);
    }
    T pop() {
        T top = std::move(h_.front());
        h_.front() = std::move(h_.back());
        h_.pop_back();
        if (!h_.empty()) percolateDown(0);
        return top;
    }
};
```

That's 40 lines for a complete priority queue. The `std::priority_queue` in the standard library is essentially this, with a few more template knobs.

### Using `std::priority_queue`

```
#include <queue>
#include <vector>

// Max-heap of ints
std::priority_queue<int> maxPQ;
maxPQ.push(5); maxPQ.push(1); maxPQ.push(9);
maxPQ.top();   // 9
maxPQ.pop();
maxPQ.top();   // 5

// Min-heap of ints
std::priority_queue<int, std::vector<int>, std::greater<int>> minPQ;

// Custom comparator on (priority, payload) pairs
using Entry = std::pair<int, std::string>;
auto cmp = [](const Entry& a, const Entry& b) { return a.first > b.first; };
```

```
std::priority_queue<Entry, std::vector<Entry>, decltype(cmp)> jobs(cmp);
jobs.push({3, "cleanup"}); jobs.push({1, "urgent"});
jobs.top();   // {1, "urgent"}   -- lower numeric priority = more urgent
```

**Gotcha**

`std::priority_queue::top` returns a *const reference*. You cannot mutate the top element in place – attempting to `const_cast` it and rewrite would break the heap invariant. If you need to change a priority, you have to pop the item and push a new one, or switch to a structure with an `increase_key` / `decrease_key` operation (indexed heap, `std::multiset`, or a hand-written heap with a side-index).

**Key idea**

An ADT separates "what a structure promises" from "how it does it." Everywhere in this course, when a problem reduces to "I keep needing the extremum of a changing set," you should reach for a priority queue – and under the hood, you can bet it's a heap.

## 5 7.5 Treaps: Heap + BST

Chapter 6 ended with a warning: a raw BST collapses on sorted input. Chapter 6.10 teased self-balancing trees (AVL, red-black) as the rigorous fix. This section shows a *lazier* fix – a fix that doesn't require tracking balance factors or recoloring – called a **treap** (tree + heap).

**Definition — Treap**

A treap is a binary tree where each node carries two keys:

- A **main key** (the data you care about), maintained in *BST order* across the tree.
- A **priority** (a random number chosen when the node is inserted), maintained in *heap order* across the tree.

Both invariants hold simultaneously: for every node $n$, main-keys follow the BST rule w.r.t. subtrees, and priorities follow the heap rule w.r.t. parent/child.

The magic observation: *given a set of (main key, priority) pairs with distinct priorities, there is exactly one tree shape that satisfies both invariants.* The highest-priority node has to be the root (heap), and among its descendants the BST ordering is determined. So if priorities are random, the tree's shape is the same tree you'd get by inserting the main keys in the *random order of their priorities.* And we already know: random-order BST insertion gives expected $O(\log n)$ height.

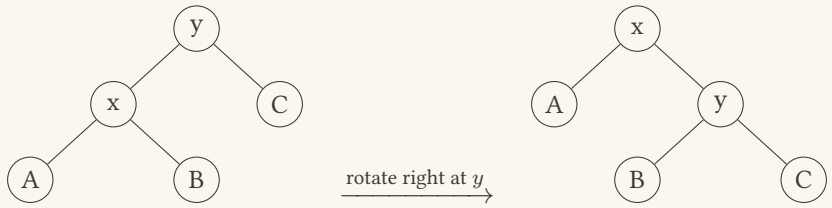
The treap is a self-balancing BST that achieves expected $O(\log n)$ operations by *randomizing its insertion order internally*, without the caller having to cooperate.

### The missing primitive: rotations

Percolate-up in a normal heap swaps parent and child. That breaks the BST ordering. Treaps need a swap-like operation that *preserves BST ordering*. That operation is the **rotation**, the same move AVL and red-black trees use.

**Definition — Right rotation**

If $y$ is a node with left child $x$, right-rotating at $y$ makes $x$ the new root of the subtree, $y$ becomes $x$'s right child, and $x$'s former right child $B$ becomes $y$'s new left child. Left rotation is the mirror image.




Check ordering: before, $A < x < B < y < C$. After, still $A < x < B < y < C$. The in-order sequence is preserved. *Heights and parent/child relationships change; BST ordering does not.*

Rotations are the universal self-balancing primitive. Once you have them, AVL, red-black, and treaps are all "use rotations to restore some auxiliary invariant after a modification." The auxiliary invariant differs per structure; the mechanics are identical.

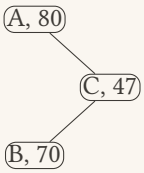
## Treap insert

1. Insert the new node at the right leaf position by main-key (pure BST insert).
2. Assign it a random priority.
3. While its priority exceeds its parent's priority, rotate so that the node moves up one level. *Rotate right* if the node is a left child; *rotate left* if it is a right child.

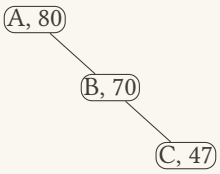
The rotation choice is forced: to move a left-child upward, only a right-rotation at the parent works; analogously for a right-child and left-rotation. Each rotation preserves BST ordering and fixes one heap-property violation. After $O(\log n)$ expected rotations, the heap invariant is restored everywhere.

Example: start with a treap and insert key $B$ (assigned priority 70):

*After BST insert of B, before rotations:*




Heap violation: parent `C`'s priority 47 is less than `B`'s 70. `B` is a left child, so rotate right at `C`:




Now `B`'s parent is `A` with priority 80 – heap holds ($80 > 70$). Stop. Treap restored.

## Treap delete

To delete a node $X$:

1. Pretend $X$'s priority is $-\infty$ (lowest possible). Now the heap property is violated at $X$: every child's priority is greater.
2. Percolate $X$ down by rotating it with the *higher-priority* child. That child becomes the new subtree root, $X$ sinks one level. Continue until $X$ has no children.
3. Unhook and delete $X$.

The "higher-priority child" rule matters: rotating with the *lower*-priority child would leave a new heap violation in place of the old one. Rotating with the higher-priority child lifts it up and fixes the violation at that level.

Example: delete node `F` from a tree where its children are `C` (priority 29) and `I` (priority 13). Assign `F`'s priority to $-\infty$. Higher-priority child is `C`, so rotate right at `F` (moving `C` up). Continue until `F` has no children, then delete.

## Cost analysis

All treap operations have **expected** cost $O(\log n)$, because the tree shape is (in expectation) the shape of a BST built from $n$ items inserted in random order – expected height $O(\log n)$.

| Operation | Expected | Worst-case |
|---|---|---|
| Search | $O(\log n)$ | $O(n)$ |
| Insert | $O(\log n)$ | $O(n)$ |
| Delete | $O(\log n)$ | $O(n)$ |

The worst case is still $O(n)$ – if your random-number generator has really bad luck, you could get a skewed tree. But the probability of this is exponentially small, so in practice treaps perform as well as AVL or red-black trees while being substantially simpler to implement.

**Key idea**

A treap doesn't guarantee balance – it *randomizes* to make imbalance improbable. This is a common pattern in computer science: don't solve the hard version (guaranteed balance, as in AVL / red-black); randomize the input to dodge worst cases (as in quicksort, hash-function-seeded hash tables, skip lists, treaps). When randomization is acceptable, it's almost always easier to implement than the deterministic version.

## Treaps vs. AVL vs. red-black

| Structure | Balance guarantee | Implementation complexity | Used by |
|---|---|---|---|
| AVL | strict $O(\log n)$ | medium | older DB indexes |
| Red-black | $O(\log n)$ (weaker balance) | high | `std::map`, Linux kernel |
| Treap | expected $O(\log n)$ | **low** | competitive programming, teaching |
| Skip list | expected $O(\log n)$ | low | Redis, LevelDB |

Treaps are rarely the choice in production standard libraries (AVL and red-black trees' deterministic guarantees matter for adversarial workloads), but they're popular in competitive-programming circles for their tiny implementation. Roughly 30 lines of C++ gets you a working treap; a working red-black tree is $10\times$ that.

**Aside**

The trick of “attach a random secondary key and let it drive structure” is deeply generalizable. Skip lists use it for levels (each node has a random “tower height”). Probabilistic data structures in databases (HyperLogLog, Count-Min sketches) use it for bucket assignments. Treaps are a clean, easy-to-visualize introduction to the technique.